Université Antonine

Faculty of Engineering
Department of Computer and Communications Engineering

Midterm – S1 2026-2027

Distributed Systems

Distributed Object-Tolerant Storage Framework

- You must develop a distributed framework that splits files into content-addressed chunks, distributes them across multiple storage nodes, keeps them available through replication and erasure coding, and exposes them through an S3-like API.
- At the end of the two sessions, you must submit a document describing your work.
- You must test your framework on two different workloads: (a) many small files (for example, 5,000 files of 1–50 KB, such as text or JSON) and (b) a few large files (for example, 10 files of 100 MB–1 GB, such as video or disk images).
- You must submit a detailed report that includes full-screen screenshots for each phase of the project, each accompanied by a paragraph explaining the steps taken in that phase. Additionally, the report must contain a flowchart illustrating each phase of the framework (write path, read path, failure and repair path) to provide a clear visual representation of the system's workflow.
- You must deliver **all files** created, used, or included (source code, Dockerfiles, configuration files, test scripts, datasets or the scripts that generate them, and benchmark results).

Objectives

- Implement a scalable storage layer distributed over at least 4 storage nodes.
- Implement data placement using consistent hashing so that adding or removing a node moves only a fraction of the data.
- Ensure fault tolerance through replication and erasure coding, with automatic detection of failed nodes and repair of lost chunks.
- Ensure data integrity through content addressing (SHA-256) and periodic scrubbing.
- Reduce storage cost through chunk-level deduplication.
- Provide APIs for storing, retrieving, listing and searching objects.

Tasks:

1. **Set up the environment**
    - Install Python and set up a virtual environment.

- Set up a version control system (GitHub/GitLab) and initialize a repository.
- Install Docker and Kubernetes.
- Allocate a separate data directory (or Docker volume) for each storage node.

2. **Develop the Storage Node Module**
   - Use FastAPI (or gRPC) to build a storage node service exposing PUT, GET, DELETE and HEAD on chunks.
   - Address every chunk by the SHA-256 hash of its content and store it on the node's local disk.
   - Verify the hash on every read and reject corrupted chunks.
   - Send a heartbeat from each node to the metadata service every few seconds (node id, free space, chunk count).
   - Run at least 4 storage nodes in separate containers.

3. **Implement the Metadata and Placement Module**
   - Split incoming files into chunks: fixed-size (for example 4 MB) first, then content-defined chunking (rolling hash, for example FastCDC) for comparison.
   - Store object metadata (bucket, key, size, version, ordered list of chunk hashes, creation date) in etcd or PostgreSQL.
   - Implement a consistent hashing ring with virtual nodes to map each chunk hash to its target nodes.
   - Implement chunk-level deduplication with reference counting, so identical chunks are stored once and deleted only when no object references them.
   - Test placement by adding and removing a node and measuring the percentage of chunks that move.

4. **Implement Redundancy and Recovery**
   - Implement replication with a configurable factor N (default 3) and quorum writes and reads ($W + R > N$).
   - Implement erasure coding (Reed–Solomon, k data + m parity, for example 4+2) using zfec or pyeclib, selectable per bucket.

- Mark a node as failed after missing heartbeats, and trigger a repair job that rebuilds its chunks on the remaining nodes.
- Implement a background scrubber that re-reads chunks, detects bit rot and repairs from a healthy copy.
- Compare the storage overhead and repair traffic of replication versus erasure coding on both workloads.

5. **Develop the API for Data Access**
   - Create a FastAPI gateway service for handling user requests.
   - Implement endpoints for:
     - Creating and listing buckets.
     - Uploading objects (streaming or multipart upload for large files).
     - Downloading objects, including byte-range requests.
     - Listing, deleting and getting the metadata of objects.
     - Searching objects by key prefix, size, date and user-defined tags.
     - Reporting cluster status (nodes, free space, chunks per node, degraded objects).
   - Implement authentication (API keys or JWT) and rate limiting.
   - Test API responses with pytest and Postman or curl.

6. **Build a Simple Web UI**
   - Use React/Vue.js to develop a basic interface.
   - Connect the UI to the FastAPI gateway for uploading, downloading and browsing objects.
   - Show the cluster state: node health, chunk distribution across nodes and repair progress.
   - Implement object search using Elasticsearch over the metadata.

Bonus

7. **Implementing Load Balancing and Fault Tolerance**
   - Set up Nginx or HAProxy as a load balancer in front of at least 2 gateway instances.

- Use Kafka or RabbitMQ to queue repair, rebalancing and garbage-collection jobs and distribute them across workers.
- Deploy storage nodes as a Kubernetes StatefulSet with persistent volumes, and configure liveness and readiness probes so failed nodes restart automatically.
- Run the metadata store as a 3-member etcd cluster (or a replicated PostgreSQL) so it is not a single point of failure.
- Set up Prometheus & Grafana to monitor throughput, latency, disk usage per node, degraded objects and repair rate.

8. Deploy the System on the Cloud

- Create Dockerfiles for each service (storage node, metadata, gateway, repair worker, UI).
- Use Docker Compose for local testing.
- Deploy the system using Kubernetes (on AWS/GCP/Azure).
- Configure auto-scaling for the gateway, and show that adding a storage node triggers rebalancing of chunks onto it.